

IoC Container

1. What is IoC?

Inversion of Control (IoC) is a design principle where the **responsibility** of **creating and managing of objects** for the classes is **transferred** from **programmer** to **Spring IoC Container**.

2. What is the Spring IoC Container?

The **Spring IoC Container** is the core component of Spring Framework.

It is responsible for:

- Creating objects (Beans)
- Injecting dependencies
- Managing bean lifecycle
- Configuring beans

Real-Life Analogy

Imagine ordering a food in restaurant.

- Without a restaurant system:

Customer → go to kitchen, Cook Food, Bring Food, server himself. (You do everything yourself)

- With a restaurant system.

Customer → Waiter (Container) → Kitchen → Food served : Customer orders food, the food is prepared and served to customer.

The same way, for the programmer.

Programmer asks for object → Spring IoC container → create objects, Inject dependencies → return ready-to-use objects to programmer.

3. Configuring the Spring IoC Container using XML

Before Spring Boot, XML was used to tell spring which classes should be managed.

Step 1: Create a Java class

```
public class Student {  
  
    public void display() {  
        System.out.println("Student Bean Created");  
    }  
}
```

This is a simple Java class.

Step 2: Create `beans.xml`

```
<beans>  
  
    <bean id="student"  
        class="com.example.Student"/>  
  
</beans>
```

Explanation

- `<bean>` → Defines an object managed by Spring.
- `id="student"` → The bean's name.
- `class` → The fully qualified class name.

When Spring starts, it reads this XML file and automatically creates a `Student` object.

Step 3: Load the IoC Container

```
ApplicationContext context =  
    new ClassPathXmlApplicationContext("beans.xml");
```

Explanation

- `ApplicationContext` creates the IoC Container.

- `ClassPathXmlApplicationContext` loads `beans.xml` .
- Spring reads the configuration and creates all the beans.

3. Defining Beans and Their Dependencies

What is a Bean?

A **Bean** is simply an object that is created and managed by the Spring IoC Container.

Example:

```
public class Engine {  
}
```

If Spring creates this object, then **Engine becomes a Spring Bean**.

What is a Dependency?

A dependency is an object that another object requires.

Example:

```
Car  
↓  
Engine
```

Here, **Car depends on Engine**.

Without Spring

```
Engine engine = new Engine();  
Car car = new Car(engine);
```

The programmer creates both objects manually.

With Spring XML

```
<bean id="engine"
      class="com.example.Engine"/>

<bean id="car"
      class="com.example.Car">

    <property name="engine"
              ref="engine"/>

</bean>
```

Explanation

- Spring creates the `Engine` bean.
- Spring creates the `Car` bean.
- Spring injects the `Engine` bean into the `Car` bean.

No manual object creation is required.

4. ApplicationContext

What is ApplicationContext?

`ApplicationContext` is the most commonly used Spring IoC Container.

It performs:

- Bean creation
- Dependency Injection
- Bean lifecycle management
- Event handling
- AOP support
- Internationalization (i18n)

```
ApplicationContext context =  
    new ClassPathXmlApplicationContext("beans.xml");
```

Retrieve a bean:

```
Student student =  
    context.getBean(Student.class);
```

The `ApplicationContext` creates the object and returns it whenever requested.

5. BeanFactory

What is BeanFactory?

It is a root interface that define fundamental IoC container responsible for creating, managing object lifecycles and dependency injection.

It performs:

- Bean creation
- Dependency Injection
- Basic bean management

Example

```
BeanFactory factory =  
    new XmlBeanFactory(new ClassPathResource("beans.xml"));
```

Retrieve a bean:

```
Student student =  
    factory.getBean(Student.class);
```

Note: `XmlBeanFactory` is deprecated in modern Spring versions but is still asked in many university exams.

6. ApplicationContext vs BeanFactory

Feature	BeanFactory	ApplicationContext
Type	Basic IoC Container	Advanced IoC Container
Bean Creation	Lazy Initialization	Eager Initialization (default)
AOP Support	Limited	Full Support
Event Handling	No	Yes
Internationalization	No	Yes
Recommended	Rarely used	Most commonly used